2. Collection Hierarchy — Quick Notes

Three Main Parts

Part	Description
Iterable	Root interface
Collection	Root interface for most collections
Map	Separate hierarchy for key-value pairs
Collections Class	Utility class with static methods

Collection vs Collections

Collection	Collections
Interface	Utility class
Root of collection hierarchy	Contains utility methods
add(), remove(), contains()	sort(), max(), min(), reverse()

Iterable Interface

Important Methods: iterator(), forEach()

Purpose

- Traversing collections
- Removing items using iterator

Collection Interface

Root Interface

Contains basic operations: add(), remove(), contains(), size(), clear()

Important

- Most collections inherit from this

List Interface

Implemented By: ArrayList, LinkedList, Vector, Stack

Additional Features

Index-based operations:

- get(index)
- set(index)

Important List Implementations

Class	Internal DS	Notes
ArrayList	Dynamic Array	Preferred
LinkedList	Doubly Linked List	Queue/Deque support
Vector	Dynamic Array	Thread-safe, legacy
Stack	Extends Vector	Legacy stack

Recommendation

- Prefer ArrayList
- Prefer ArrayDeque over Stack

Queue Interface

FIFO Structure

Important Methods: offer(), poll()

Implemented By: PriorityQueue, LinkedList

PriorityQueue

Internal DS: Heap, MinHeap / MaxHeap

Important: Removes items based on priority

Why Interfaces Are Important?

Advantage

Write generic code using interfaces.

Example: List<Integer> list = new ArrayList<>();

Later can change to: List<Integer> list = new LinkedList<>();

Without changing business logic.

Deque Interface

Features

Insert/delete from both ends.

Important Methods

- offerFirst()
- offerLast()
- pollFirst()
- pollLast()
- peekFirst()
- peekLast()

Implemented By: LinkedList, ArrayDeque

Set Interface

Features

- Distinct elements only
- No duplicates

Implemented By: HashSet, LinkedHashSet, TreeSet

SortedSet

Implemented By: TreeSet

Important

Elements must be comparable.

Uses:

- Self-balancing BST

Map Hierarchy

Purpose

Stores key-value pairs.

Use Cases

- StudentId → Student Object
- Item → Frequency

Map Interface

Important Methods

- put()
- get()
- containsKey()
- containsValue()
- clear()
- size()

Important

- Map does NOT inherit Collection

SortedMap

Implemented By

- TreeMap

Important

Keys must be comparable.

Map Implementations

Class	Internal DS	Features
HashMap	Hashing	Fast lookup
TreeMap	Red-Black Tree	Sorted keys
LinkedHashMap	Hash + Linked List	Insertion order

Package

import java.util.*;

All collections are inside:

- `java.util`

Primitive Types Not Supported

❌ Not allowed: ArrayList<int>

✅ Use Wrapper Classes: ArrayList<Integer>

Wrapper Examples

- Integer
- Character
- Double

Important Point About null

Collections heavily use:

- `null`

Example: poll()

Returns:

- removed item
- `null` if queue empty

Primitive types cannot support null.